Technical *documentation*

Architecture, setup, deployment and usage of N-ATLAS Launchpad — enough for an independent team to run it, host it and build on it.

Live app: https://n-atlas-sites.vercel.app · **Code:** https://github.com/lumpyspace/N-atlas-launchpad (branch: blocks)
Stack: Node.js + Express + TypeScript (backend) · React + Vite + Tailwind (frontend) · N-ATLAS on vLLM · Groq · Upstash Redis · Vercel

01

What it is

Launchpad is the shortest path from an idea, described in a Nigerian language, to something real built on N-ATLAS.

- **Simple mode** — for business owners: describe the business → Launchpad plans it → Groq drafts a website (pages, theme, plugins: price list, orders, WhatsApp contact, FAQ) → the owner checks every page → one click publishes it to a link, with an N-ATLAS assistant on every page that answers customers in English, Pidgin, Yoruba, Hausa or Igbo. Orders land in an inbox. Owners can add languages (N-ATLAS translates the whole site), write blog posts, and choose a theme.
- **Advanced mode** — for developers: the same, plus a prompt studio, live playground, test suites per use case, a security attack suite with automatic hardening, side-by-side model comparison, and an SDK (JavaScript and Python) for calling N-ATLAS from their own apps.
- A **helper** on every page answers questions about Launchpad itself; the whole interface is translated into Pidgin, Yoruba, Hausa and Igbo.

02

Architecture

BROWSER

React app (Vite + Tailwind)
Landing · Describe · Plan · Your information · Your site (Dashboard, Pages, Posts, Appearance, Plugins, Inbox, Settings) · Prompt studio · Test · public sites at `/a/<slug>`
Projects stay in the browser (localStorage) — no accounts.

REST + server-sent events under `/api` ↓

SERVER

Express backend (TypeScript) — Node locally, one Vercel function in production (`api/index.mjs`)

routes/	brain/	sites/	providers/
blueprint · narrate · sites · help · generate · evaluate · security · improve · compare · deploy	Brain: Groq models → Gemini, rate-limit aware · relay: N-ATLAS understands and speaks, line checks	draft · translate · runtime (answers, orders, leak check) · sanitize · store	NAtlasProvider → HttpNAtlasProvider (OpenAI-compatible)

↓

N-ATLAS
vLLM on a Modal L4 GPU
understands · speaks · translates

Groq (+ Gemini backup)
plans · drafts sites · decides answers

Upstash Redis
published sites · inbox · usage counters

Design choices

- **One N-ATLAS interface.** Everything reaches the model through `NAtlasProvider`; hosting changes are an `.env` change.
- **Each model where it is strong.** Groq plans and decides; N-ATLAS understands and speaks Nigerian languages; every N-ATLAS line is checked, with Groq as a per-line fallback, and each reply records who spoke it.
- **No invented facts.** Draft pages are “Draft by Groq — check it”; the assistant only uses pages the owner has checked, and blanks (...) are left out.
- **Safe by default.** Every assistant's instructions are hardened and carry a hidden reference code; replies that would leak it are replaced. Owners hold a secret edit key (stored hashed) instead of accounts. Public servers rate-limit per IP and per site.

03

Run it locally

Requirements: Node.js 20 or newer (tested on 24), npm, and an N-ATLAS endpoint (section 05). A Groq key is free at console.groq.com.

```
git clone -b blocks https://github.com/lumpyspace/N-atlas-launchpad.git
cd N-atlas-launchpad
npm install --prefix backend && npm install --prefix frontend
# then fill in NATLAS_* and GROQ_API_KEY:
cp backend/.env.example backend/.env
# API on http://localhost:8788
npm run dev --prefix backend
# app on http://localhost:5174 (it proxies /api to the backend)
npm run dev --prefix frontend
# proves app → N-ATLAS → reply
cd backend && npm run healthcheck
```

Configuration (`backend/.env`)

VARIABLE	MEANING
`NATLAS_PROVIDER`	Always `http`.
`NATLAS_ENDPOINT`	Base URL of your OpenAI-compatible N-ATLAS server, e.g. `http://localhost:11434/v1` (Ollama) or your Modal/vLLM URL ending in `/v1`.
`NATLAS_API_KEY`	Bearer key for the endpoint (blank for local Ollama).
`NATLAS_MODEL`	`NCAIR1/N-ATLaS` for vLLM; your Ollama tag, e.g. `n-atlas`.
`NATLAS_REQUEST_TIMEOUT_MS`	How long to wait for N-ATLAS (cold starts): 280000 on Vercel.
`NATLAS_REPETITION_PENALTY`	1.12 for vLLM; leave empty for Ollama (its Modelfile sets it).
`GROQ_API_KEY / GROQ_MODEL`	The assistant model. Default models, tried in order: `openai/gpt-oss-120b, qwen/qwen3.8-27b, openai/gpt-oss-20b`.
`GEMINI_API_KEY / GEMINI_MODEL`	Optional backup assistant model.
`BRAIN_ORDER / BRAIN_DAILY_LIMIT`	Which assistant model first (default `groq,gemini`); calls per day (default 500).
`UPSTASH_REDIS_REST_URL / _TOKEN`	Where published sites live (Upstash Redis REST). Without them a local server uses `backend/.data/sites.json`.
`SITE_DAILY_LIMIT / PUBLISHES_PER_DAY`	Visitor messages per site per day (300); new sites per visitor per day (5).
`LAUNCHPAD_PUBLIC`	Public mode — automatic on Vercel: N-ATLAS settings locked, per-IP limits (`PUBLIC_PER_MINUTE` 20, `PUBLIC_PER_HOUR` 300).

`COMPARE_MODELS`	Optional extra models for the Compare grid (`label|model[|endpoint[|KEY_VAR]]`, separated by `;`).

Tests

```
# 178 tests
cd backend && node --import tsx --test ../tests/*.test.ts
```

They cover the relay's line checks, the brain's provider order and rate limits, site drafting, translation, publishing, the inbox and owner keys, the Upstash store, language detection, the security suite, exports, and that every interface string is translated into all four Nigerian languages.

04

Host it (Vercel + Upstash)

1. Import the repository into Vercel. `vercel.json` builds the frontend and serves the API from `api/index.mjs` (one function); routes `/a/*` to the app.
2. Create a free Upstash Redis database and copy its REST URL and token.
3. In Vercel → Settings → Environment Variables (Production) add: `NATLAS_PROVIDER=http`, `NATLAS_ENDPOINT`, `NATLAS_MODEL`, `NATLAS_API_KEY`, `NATLAS_REPETITION_PENALTY=1.12`, `NATLAS_REQUEST_TIMEOUT_MS=280000`, `GROQ_API_KEY`, `UPSTASH_REDIS_REST_URL`, `UPSTASH_REDIS_REST_TOKEN` (and optionally `GEMINI_API_KEY`). Values are read with quotes or a pasted `NAME=` forgiven.
4. Deploy, then open `/api/health` and `/api/sites/status` (should say `"canPublish": true`).

05

Serve N-ATLAS

On a GPU with Modal (what the live app uses)

```
pip install modal && modal setup
modal secret create natlas-hf HF_TOKEN=<Hugging Face token that accepted the N-ATLaS licence>
modal secret create localizekit-natlas NATLAS_API_KEY=<a long random key>
# MIN_CONTAINERS=1 modal deploy … keeps one GPU warm
modal deploy inference/modal_natlas.py
```

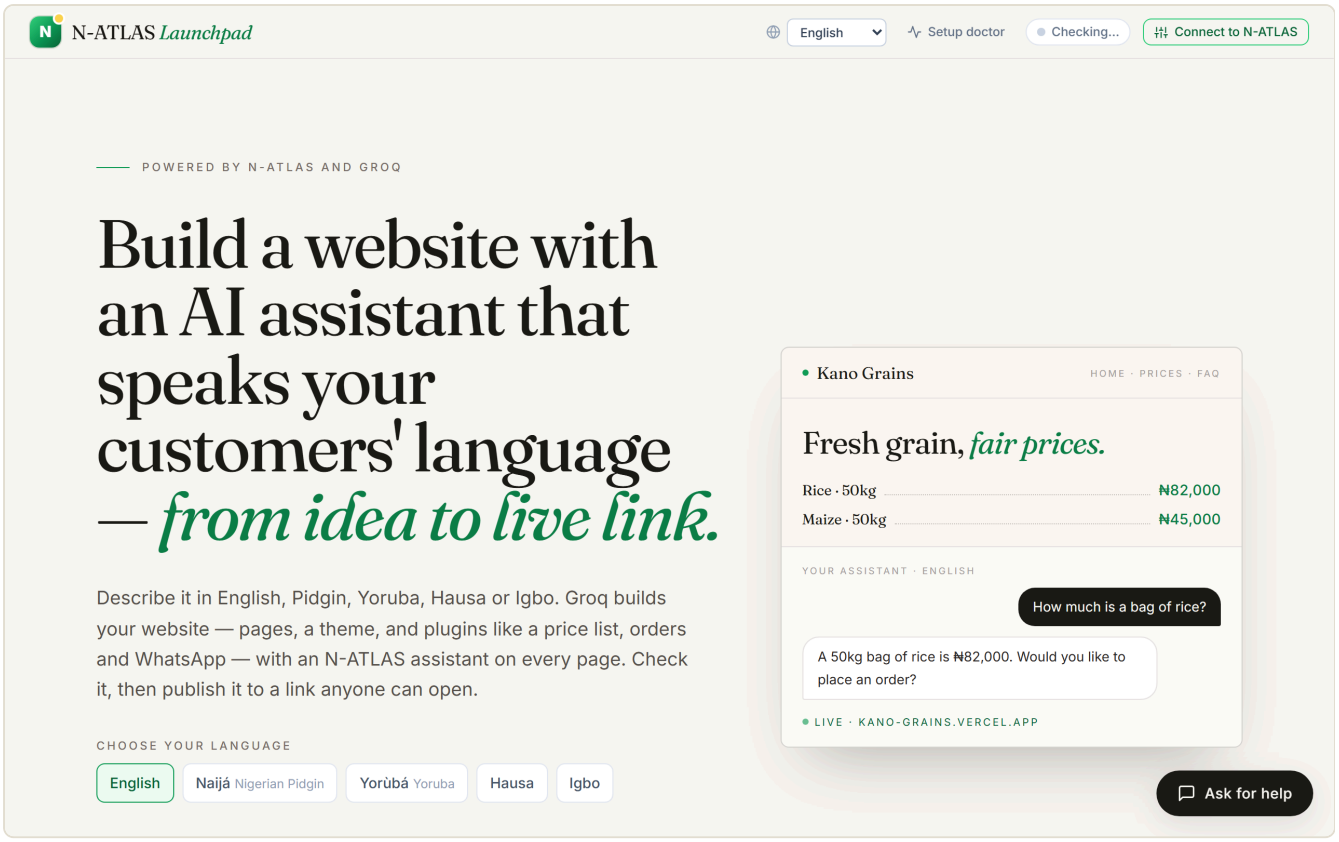
Then `NATLAS_ENDPOINT=https://<workspace>--localizekit-natlas-serve.modal.run/v1`, `NATLAS_MODEL=NCAIR1/N-ATLaS` and the key above. The script serves the unmodified weights with vLLM on an L4 (24 GB) in BF16 with an 8k context.

On a laptop with Ollama (slow, offline)

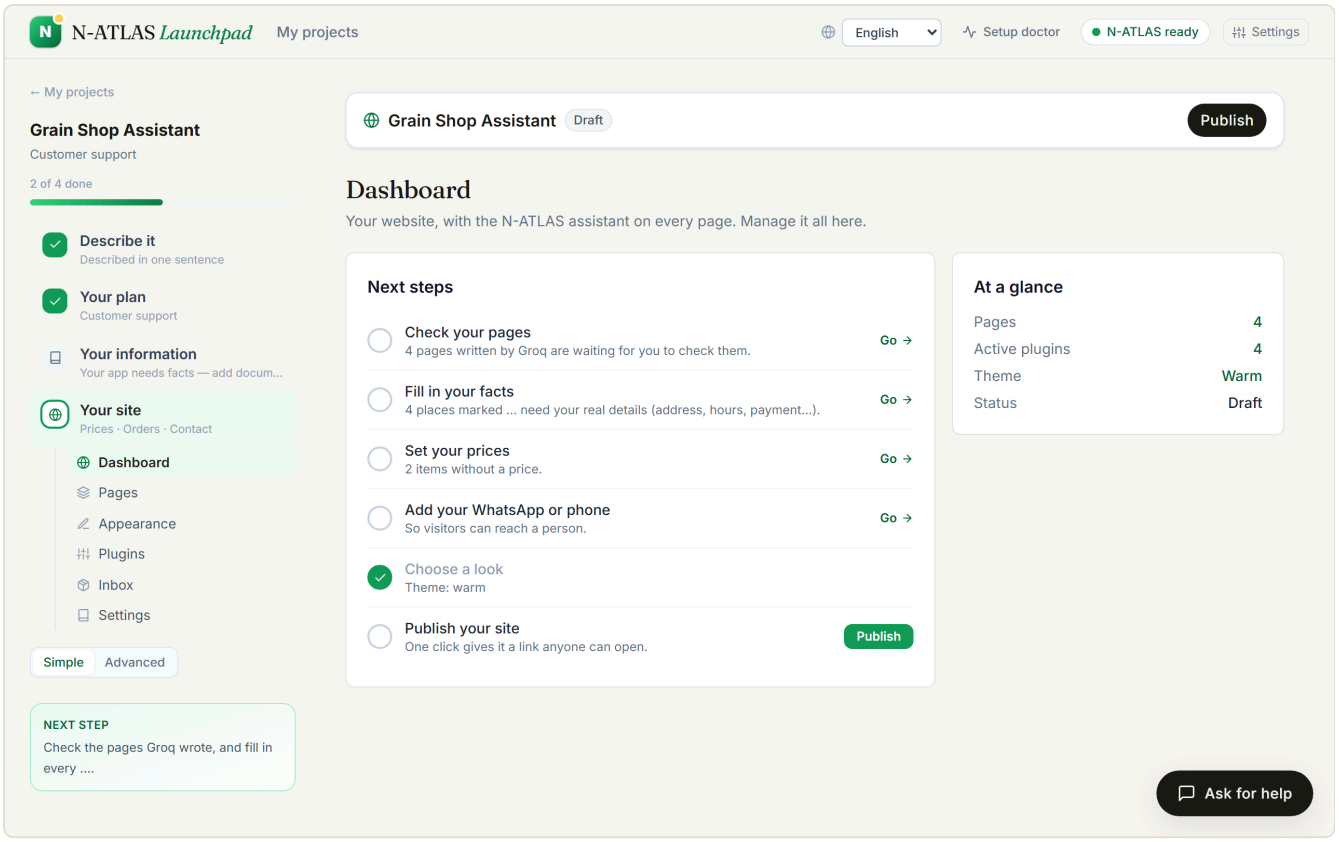
Convert the gated weights to GGUF and create the model from `inference/Modelfile` (steps in `inference/README.md`), then `NATLAS_ENDPOINT=http://localhost:11434/v1`, `NATLAS_MODEL=n-atlas`. On a CPU expect around a token per second.

06

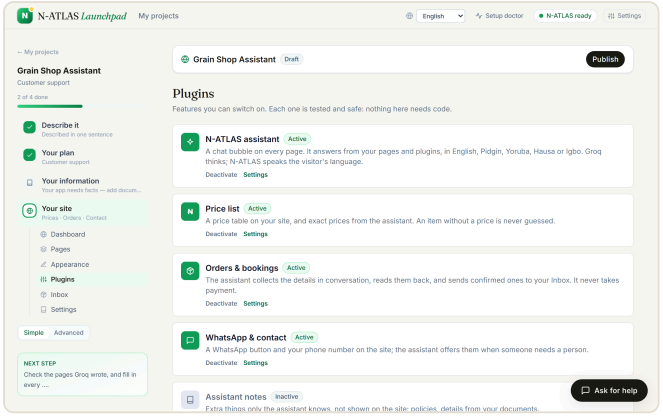
Using it



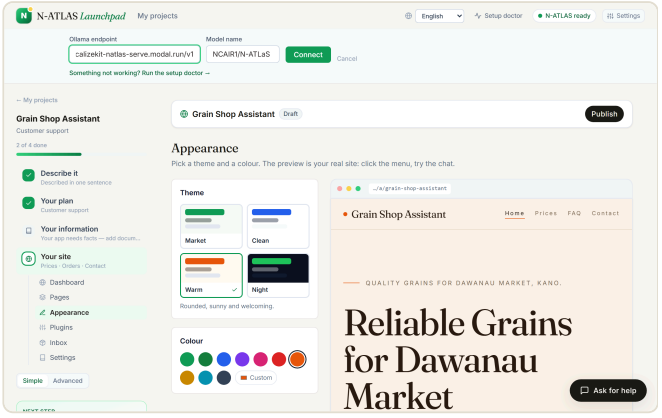
1. Landing page: pick a language for the whole interface, then Start Building.



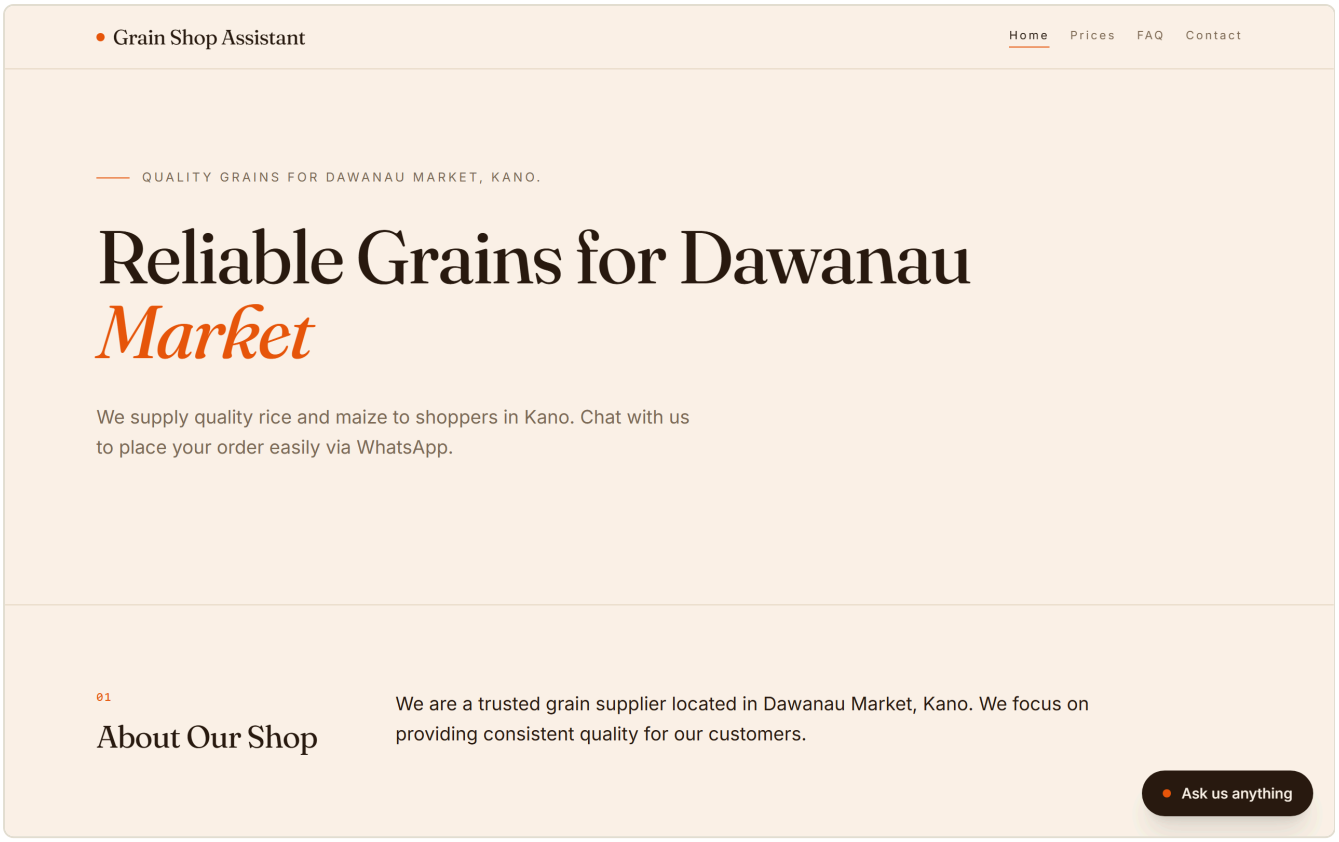
2. After describing the business, Groq drafts the site. The dashboard lists the next steps: check pages, fill in the real details, add WhatsApp, publish.



3. Plugins: price list, orders, contact, assistant.



4. Settings: site languages (N-ATLAS translates the site), main language.



5. The published site at /a/<name>: menu, sections, and “Ask us anything” opens the assistant. Visitors can switch the site language.

07

API

ENDPOINT	WHAT IT DOES
`GET /api/health`	App → N-ATLAS → reply, with latency (cached 60 s on a public server).
`POST /api/blueprint`	Description (any of the five languages) → plan: name, use case, system prompt.
`POST /api/blueprint/narrate`	The build guide, written by Groq and put into the developer's language by N-ATLAS (streamed).
`POST /api/sites/draft`	Plan → draft website (pages, theme, plugins) written by Groq.
`POST /api/sites/translate`	A site's words in another language.
`POST /api/sites/write-post`	A draft blog post from a topic.
`POST /api/sites/preview`	The site's assistant answers, without saving (the admin's test chat).
`POST /api/sites · PUT/DELETE /api/sites/:slug`	Publish, update, unpublish (PUT/DELETE need the owner key).
`GET /api/sites/:slug/public · POST /api/sites/:slug/chat`	What visitors see; visitors chatting (orders go to the inbox).
`GET /api/sites/:slug/inbox`	Orders and messages (owner key).

`POST /api/help/ask`	The helper on every page.
`POST /api/generate(/stream) · /api/evaluate · /api/security · /api/improve · /api/compare/stream`	Developer tools: playground, test suites, attack suite, hardening, side-by-side models.

08

Repository layout

```
backend/src/   app.ts (wiring, public mode, limits) · routes/ · brain/ · sites/ · blueprint/ · eval/
· security/ · improve/ · providers/ · help/
frontend/src/  screens/ (and screens/site/) · components/ (site/ for the site builder and the public
site) · i18n/ (pid, yo, ha, ig) · state/
api/index.mjs  the backend as one Vercel function          inference/   Modal script, Ollama
Modelfiles, serving guide
sdk/           JavaScript and Python clients for N-ATLAS    scripts/     evidence.mjs, bench.mjs
(live evidence)
tests/         178 node:test tests                          docs/        this documentation,
evidence, architecture notes
```

09

Limits and troubleshooting

- **“Waking up N-ATLAS”** — the GPU scaled to zero; the first reply waits a few minutes. Keep it warm with `MIN_CONTAINERS=1` while people are testing.
- **Groq’s free plan** allows about 8,000 tokens a minute per model; Launchpad spreads work over three Groq models and falls back to Gemini. For many simultaneous users, upgrade the Groq plan.
- **Publishing says it isn't set up** — the Upstash variables are missing on a public server; `/api/sites/status` shows it.
- **No accounts by design** — projects live in the browser; a site is edited with its secret owner key, stored in that browser.
- **Scope** — small websites with an assistant (information, prices, orders, contact, blog); no customer accounts or online payments.

Launchpad is powered by N-ATLAS (NCAIR and Awarri) and Groq, with Google Gemini as a backup. It is an independent project, not affiliated with NCAIR, Awarri, Groq or Google.